

Level 1: Basics

1. Retrieve customer names and emails for email marketing

This helps the marketing team extract basic customer contact details for campaigns.

Query Answer: `SELECT name, email FROM customers;`

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	email			
▶	Thomas Owens	user1@example.com			
	Charles Grant	user2@example.com			
	Kaitlin Richards	user3@example.com			
	Christina Williams	user4@example.com			
	David Allen	user5@example.com			
	Mark Duke	user6@example.com			
	Briana Wright	user7@example.com			
	John Bryan	user8@example.com			
	Jason Thompson	user9@example.com			
	Shawn Hill	user10@example.com			
	Walter Jenkins	user11@example.com			
	Mary Knight	user12@example.com			
	Leslie Wilson	user13@example.com			
	Deborah Arias	user14@example.com			
	Austin Flores	user15@example.com			
	Amy Landry	user16@example.com			
	Randy Mooney	user17@example.com			
	Jeffrey Bray	user18@example.com			
	Amanda Bright	user19@example.com			
	Megan Lee	user20@example.com			
	Jessica Zamora	user21@example.com			
	Peter Phillips	user22@example.com			
	Kathryn Mathews	user23@example.com			
	Brandy Wright	user24@example.com			
	Cindy Hart	user25@example.com			
	Victoria Hall	user26@example.com			
	Adrienne Green	user27@example.com			
	Joseph Stuart	user28@example.com			
	Kara Zavala	user29@example.com			
	Victoria Acevedo	user30@example.com			

2. View complete product catalog with all available details

The product manager may want to review all product listings in one go.

Query: SELECT * FROM products;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	product_id	name	category	price	stock_quantity	added_on
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	product_id	name	category	price	stock_quantity	added_on
	22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
	23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
	24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
	27	Answer But	Home	1016.68	90	2025-04-06 21:54:11
	28	East Foot	Toys	2414.93	128	2025-05-11 16:02:29
	29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
	30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	32	Factor Where	Clothing	2295.14	167	2023-07-07 00:59:39
	33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
	36	Television Stock	Clothing	421.73	115	2024-04-23 23:54:15
	37	Between Up	Toys	1429.45	61	2025-01-27 01:30:37
	38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
	39	Data Add	Books	4063.38	62	2025-05-26 23:39:16
	40	Toward Minute	Clothing	857.85	136	2025-04-04 07:04:21
	41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
	42	Weight Enter	Clothing	3875.18	60	2024-01-26 08:21:55

43	Least Green	Toys	1398.26	81	2024-04-10 15:23:33
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
45	Suffer We	Electronics	3657.43	113	2024-07-25 12:40:57
46	Age Treatment	Home	3728.83	160	2023-12-01 10:51:52
47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
49	Rest Something	Books	3077.82	181	2023-08-09 03:42:42
50	Stock Article	Home	834.75	131	2024-11-27 15:58:32

3. List all unique product categories

Useful for analyzing the range of departments or for creating filters on the website.

Query: `SELECT DISTINCT category FROM products;`



category
Home
Clothing
Electronics
Toys
Books

4. Show all products priced above ₹1,000

This helps identify high-value items for premium promotions or pricing strategy reviews.

Query: `SELECT * FROM products WHERE price > 1000;`

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	product_id	name	category	price	stock_quantity	added_on
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
	22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
	23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
	24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	product_id	name	category	price	stock_quantity	added_on
	27	Answer But	Home	1016.68	90	2025-04-06 21:54:11
	28	East Foot	Toys	2414.93	128	2025-05-11 16:02:29
	29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
	30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	32	Factor Where	Clothing	2295.14	167	2023-07-07 00:59:39
	33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
	37	Between Up	Toys	1429.45	61	2025-01-27 01:30:37
	38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
	39	Data Add	Books	4063.38	62	2025-05-26 23:39:16
	41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
	42	Weight Enter	Clothing	3875.18	60	2024-01-26 08:21:55
	43	Least Green	Toys	1398.26	81	2024-04-10 15:23:33
	45	Suffer We	Electronics	3657.43	113	2024-07-25 12:40:57
	46	Age Treatment	Home	3728.83	160	2023-12-01 10:51:52
	48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
	49	Rest Something	Books	3077.82	181	2023-08-09 03:42:42

5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

A merchandising team might need this to create a mid-tier pricing campaign.

Query:

```
SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

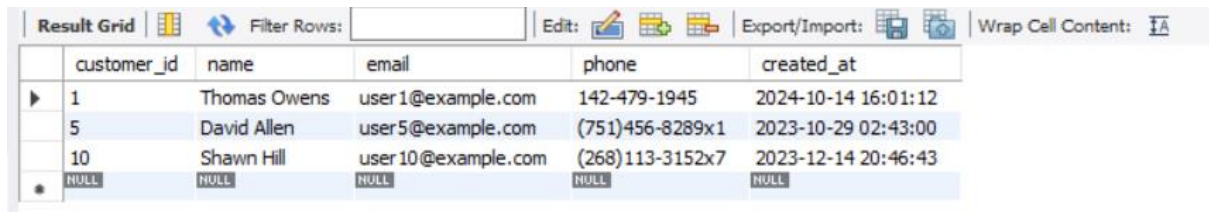
Wrap Cell Content:

	product_id	name	category	price	stock_quantity	added_on
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
	22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
	23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
	24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
	28	East Foot	Toys	2414.93	128	2025-05-11 16:02:29
	29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
	30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	32	Factor Where	Clothing	2295.14	167	2023-07-07 00:59:39
	33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
	35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
	38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
	39	Data Add	Books	4063.38	62	2025-05-26 23:39:16
	41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
	42	Weight Enter	Clothing	3875.18	60	2024-01-26 08:21:55
	45	Suffer We	Electronics	3657.43	113	2024-07-25 12:40:57
	46	Age Treatment	Home	3728.83	160	2023-12-01 10:51:52
	48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
	49	Rest Something	Books	3077.82	181	2023-08-09 03:42:42

6. Fetch data for specific customer IDs (e.g., from loyalty program list)

This is used when customer IDs are pre-selected from another system.

Query: `SELECT * FROM customers WHERE customer_id IN (1, 5, 10);`



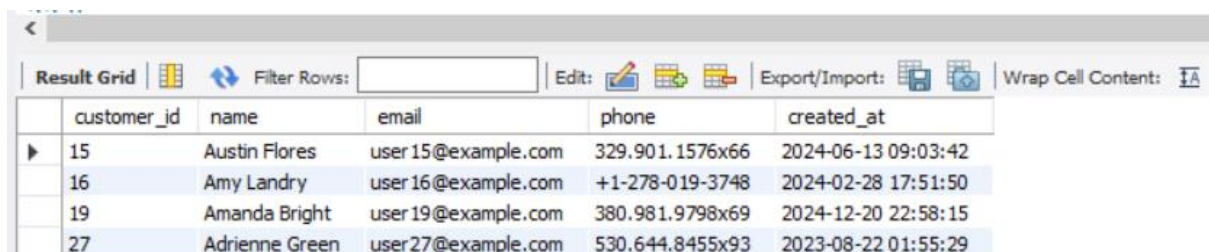
The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of a query filtering for customer IDs 1, 5, and 10. The columns are customer_id, name, email, phone, and created_at. The data is as follows:

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43

7. Identify customers whose names start with the letter 'A'

Used for alphabetical segmentation in outreach or app display.

Query: `SELECT * FROM customers WHERE name LIKE 'A%';`



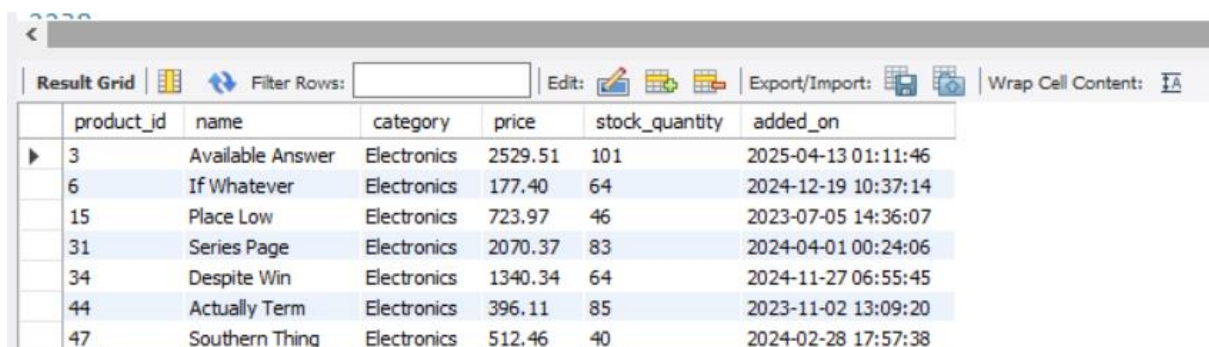
The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of a query filtering for customer names starting with 'A'. The columns are customer_id, name, email, phone, and created_at. The data is as follows:

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29

8. List electronics products priced under ₹3,000

Used by merchandising or frontend teams to showcase budget electronics.

Query: `SELECT * FROM products WHERE category = 'Electronics' AND price < 3000;`



The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of a query filtering for electronics products priced under ₹3,000. The columns are product_id, name, category, price, stock_quantity, and added_on. The data is as follows:

product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38

9. Display product names and prices in descending order of price

This helps teams easily view and compare top-priced items.

Query: SELECT name, price FROM products ORDER BY price DESC;

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	price			
▶	Response Indeed	4897.36			
	Population Social	4813.68			
	Development System	4801.78			
	Any Question	4759.28			
	Fire Often	4734.89			
	Natural Network	4722.66			
	Build High	4707.14			
	Serious Recognize	4523.10			
	Study Total	4413.68			
	Real Source	4398.66			
	Involve Officer	4244.09			
	Group But	4180.23			
	Every Amount	4173.60			
	Move Small	4168.08			
	Drop Remember	4160.45			
	Data Add	4063.38			
	Action Ask	4017.01			
	Weight Enter	3875.18			
	Bed Including	3845.31			
	Age Treatment	3728.83			
	Some Them	3673.86			

Result Grid				Filter Rows:	Export: 	Wrap Cell Content: 
	name	price				
	Suffer We	3657.43				
	South Free	3543.58				
	Movement Future	3502.62				
	Assume Serve	3437.81				
	Rest Something	3077.82				
	Future Sort	3043.67				
	Available Answer	2529.51				
	Everything Plant	2496.68				
	East Foot	2414.93				
	Factor Where	2295.14				
	Life Series	2208.99				
	Full West	2112.33				
	Series Page	2070.37				
	Build Her	1852.64				
	Between Up	1429.45				
	Least Green	1398.26				
	Despite Win	1340.34				
	Special Fact	1094.18				
	Answer But	1016.68				
	Common Study	985.19				
	Toward Minute	857.85				

	Stock Article	834.75
	Place Low	723.97
	Plant No	639.43
	Southern Thing	512.46
	Television Stock	421.73
	Actually Term	396.11
	Listen Development	296.17
	If Whatever	177.40

10. Display product names and prices, sorted by price and then by name

The merchandising or catalog team may want to list products from most expensive to cheapest. If multiple products have the same price, they should be sorted alphabetically for clarity on storefronts or printed catalogs.

Query: `SELECT name, price FROM products ORDER BY price DESC, name ASC;`

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	price			
▶	Response Indeed	4897.36			
	Population Social	4813.68			
	Development System	4801.78			
	Any Question	4759.28			
	Fire Often	4734.89			
	Natural Network	4722.66			
	Build High	4707.14			
	Serious Recognize	4523.10			
	Study Total	4413.68			
	Real Source	4398.66			
	Involve Officer	4244.09			
	Group But	4180.23			
	Every Amount	4173.60			
	Move Small	4168.08			
	Drop Remember	4160.45			
	Data Add	4063.38			
	Action Ask	4017.01			
	Weight Enter	3875.18			
	Bed Including	3845.31			
	Age Treatment	3728.83			
	Some Them	3673.86			

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	name	price
	Suffer We	3657.43
	South Free	3543.58
	Movement Future	3502.62
	Assume Serve	3437.81
	Rest Something	3077.82
	Future Sort	3043.67
	Available Answer	2529.51
	Everything Plant	2496.68
	East Foot	2414.93
	Factor Where	2295.14
	Life Series	2208.99
	Full West	2112.33
	Series Page	2070.37
	Build Her	1852.64
	Between Up	1429.45
	Least Green	1398.26
	Despite Win	1340.34
	Special Fact	1094.18
	Answer But	1016.68
	Common Study	985.19
	Toward Minute	857.85
	Stock Article	834.75
	Place Low	723.97
	Plant No	639.43
	Southern Thing	512.46
	Television Stock	421.73
	Actually Term	396.11
	Listen Development	296.17
	If Whatever	177.40

Level 2: Filtering and Formatting

1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)

Used to identify orphaned orders or test data where customer_id is not linked.

```
SELECT o.*  
FROM orders o  
LEFT JOIN customers c ON o.customer_id = c.customer_id  
WHERE c.customer_id IS NULL;
```

Result Grid			Filter Rows:	Export: 	Wrap Cell Content: 
	order_id	customer_id	order_date	status	total_amount

2. Display customer names and emails using column aliases for frontend readability

Useful for feeding into frontend displays or report headings that require user-friendly labels.

```
SELECT name AS CustomerName, email AS CustomerEmail
FROM customers;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	CustomerName	CustomerEmail			
▶	Thomas Owens	user1@example.com			
	Charles Grant	user2@example.com			
	Kaitlin Richards	user3@example.com			
	Christina Williams	user4@example.com			
	David Allen	user5@example.com			
	Mark Duke	user6@example.com			
	Briana Wright	user7@example.com			
	John Bryan	user8@example.com			
	Jason Thompson	user9@example.com			
	Shawn Hill	user10@example.com			
	Walter Jenkins	user11@example.com			
	Mary Knight	user12@example.com			
	Leslie Wilson	user13@example.com			
	Deborah Arias	user14@example.com			
	Austin Flores	user15@example.com			
	Amy Landry	user16@example.com			
	Randy Mooney	user17@example.com			
	Jeffrey Bray	user18@example.com			
	Amanda Bright	user19@example.com			
	Megan Lee	user20@example.com			
	Jessica Zamora	user21@example.com			
	Peter Phillips	user22@example.com			
	Kathryn Mathews	user23@example.com			
	Brandy Wright	user24@example.com			
	Cindy Hart	user25@example.com			
	Victoria Hall	user26@example.com			
	Adrienne Green	user27@example.com			
	Joseph Stuart	user28@example.com			
	Kara Zavala	user29@example.com			
	Victoria Acevedo	user30@example.com			

3. Calculate total value per item ordered by multiplying quantity and item price

This can help generate per-line item bill details or invoice breakdowns.

```
SELECT order_id, quantity * price AS total_item_value
FROM order_items;
```

it gives no output because order_items table does NOT exist in your database

4. Combine customer name and phone number in a single column

Used to show brief customer summaries or contact lists.

```
SELECT CONCAT(name, ' - ', phone) AS customer_contact
FROM customers;
```

Result Grid			 Filter Rows:	Export: 	Wrap Cell Content: 
	customer_contact				
▶	Thomas Owens - 142-479-1945				
	Charles Grant - 9153947511				
	Kaitlin Richards - 2073473421				
	Christina Williams - 586-605-5061x06				
	David Allen - (751)456-8289x1				
	Mark Duke - (144)957-2811				
	Briana Wright - 223-833-9635				
	John Bryan - 045.568.0798x27				
	Jason Thompson - 1862659420				
	Shawn Hill - (268)113-3152x7				
	Walter Jenkins - 536-329-0817x71				
	Mary Knight - 361-636-3802				
	Leslie Wilson - +1-256-261-1984				
	Deborah Arias - 811-821-2144x97				
	Austin Flores - 329.901.1576x66				
	Amy Landry - +1-278-019-3748				
	Randy Mooney - (158)927-7313x3				
	Jeffrey Bray - 164.962.0222x92				
	Amanda Bright - 380.981.9798x69				
	Megan Lee - 7044478333				
	Jessica Zamora - 177-816-7773x00				
	Peter Phillips - 389-656-1695				
	Kathryn Mathews - 001-707-722-178				
	Brandy Wright - 853.520.8915x61				
	Cindy Hart - 617.574.8421x41				
	Victoria Hall - 926-840-0016				
	Adrienne Green - 530.644.8455x93				
	Joseph Stuart - 363-045-4287				
	Kara Zavala - 434-985-5776x61				
	Victoria Acevedo - +1-325-063-7244				

5. Extract only the date part from order timestamps for date-wise reporting

Helps group or filter orders by date without considering time.

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	order_date			
▶	2025-03-02			
	2024-10-09			
	2025-05-08			
	2024-09-19			
	2025-04-08			
	2024-10-25			
	2024-07-29			
	2024-07-30			
	2025-06-10			
	2025-02-16			
	2025-03-11			
	2025-01-15			
	2025-05-12			
	2024-09-24			
	2024-07-18			
	2024-09-09			
	2024-08-19			
	2024-06-15			
	2024-07-06			

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	order_date			
	2024-08-24			
	2025-03-12			
	2025-02-21			
	2025-03-02			
	2024-06-19			
	2024-07-24			
	2025-02-16			
	2024-11-22			
	2024-10-19			
	2025-02-17			
	2024-10-18			
	2024-12-30			
	2024-10-20			
	2025-04-27			
	2025-05-09			
	2024-11-27			
	2024-07-08			
	2025-05-10			
	2024-07-05			

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	order_date
	2024-12-29
	2024-11-18
	2025-01-15
	2024-09-23
	2025-03-05
	2025-02-07
	2024-08-20
	2024-11-11
	2024-06-18
	2024-10-14
	2025-05-21
	2024-07-12
	2024-12-20
	2025-02-27
	2024-08-24
	2024-08-29
	2025-02-07
	2024-06-20
	2024-10-16

... I am pasting a few outputs because the full output is extremely lengthy.

Result Grid

6. List products that do not have any stock left

This helps the inventory team identify out-of-stock items.

```
SELECT * FROM products WHERE stock_quantity = 0;
```

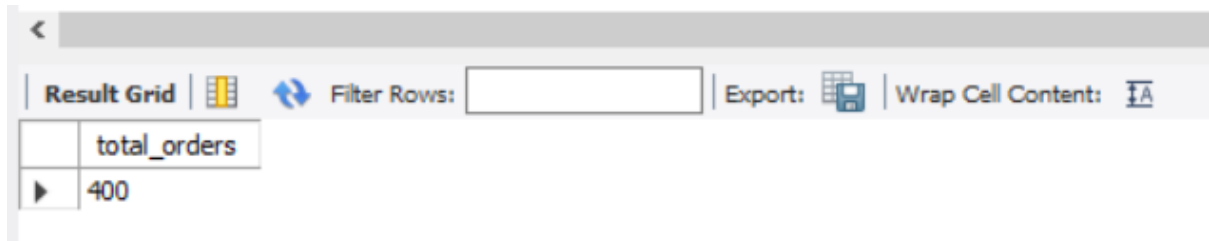
Result Grid			Filter Rows:	Edit:			Export/Import:		Wrap Cell Content:
	product_id	name	category	price	stock_quantity	added_on			
*	NULL	NULL	NULL	NULL	NULL	NULL			

Level 3: Aggregations

1. Count the total number of orders placed

Used by business managers to track order volume over time.

Query: `SELECT COUNT(*) AS total_orders FROM orders;`



The screenshot shows a database query result grid. The header row is labeled 'total_orders'. The first data row shows the value '400'. The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' toggle.

	total_orders
▶	400

2. Calculate the total revenue collected from all orders

This gives the overall sales value.

Query: `SELECT SUM(total_amount) AS total_revenue FROM orders;`



The screenshot shows a database query result grid. The header row is labeled 'total_revenue'. The first data row shows the value '6960973.66'. The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' toggle.

	total_revenue
▶	6960973.66

3. Calculate the average order value

Used for understanding customer spending patterns.

Query: `SELECT AVG(total_amount) AS avg_order_value FROM orders;`



The screenshot shows a database query result grid. The header row is labeled 'avg_order_value'. The first data row shows the value '17402.434150'. The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' toggle.

	avg_order_value
▶	17402.434150

4. Count the number of customers who have placed at least one order

This identifies active customers.

Query: `SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders;`



The screenshot shows a database query result grid. The toolbar includes a back arrow, 'Result Grid' button, a grid icon, a refresh icon, 'Filter Rows:' text with an input field, 'Export:' button with a download icon, and 'Wrap Cell Content:' button with a text icon. The result table has two columns: 'active_customers' and a value '30'.

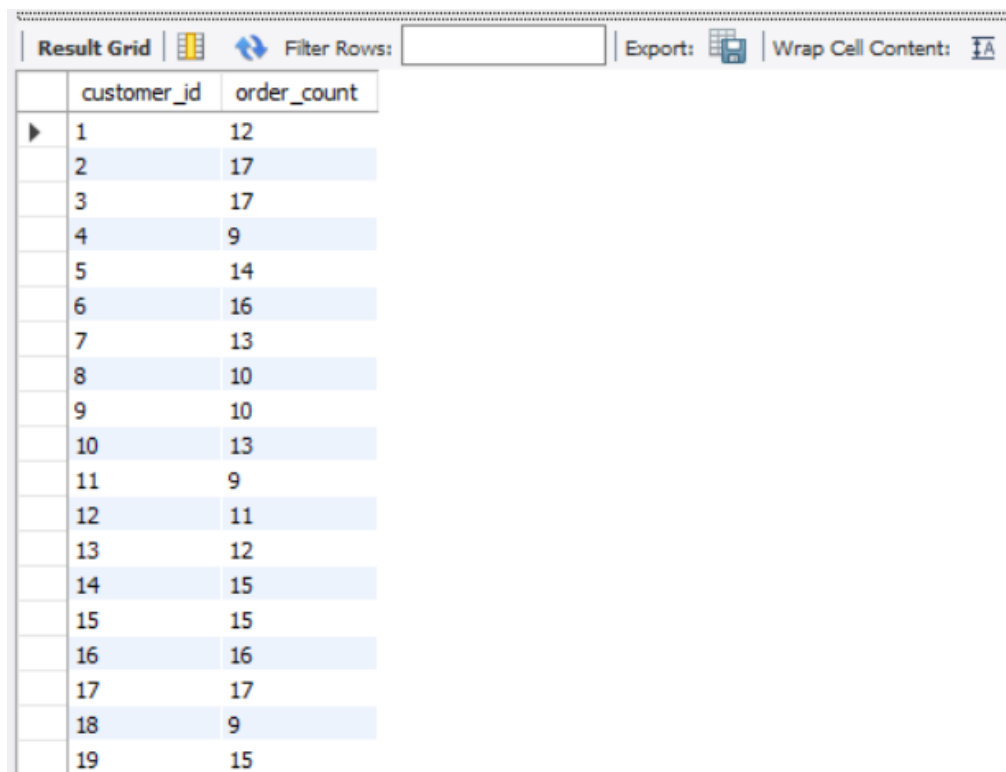
	active_customers
▶	30

5. Find the number of orders placed by each customer

Helpful for identifying top or repeat customers.

Query:

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```



The screenshot shows a database query result grid. The toolbar includes a back arrow, 'Result Grid' button, a grid icon, a refresh icon, 'Filter Rows:' text with an input field, 'Export:' button with a download icon, and 'Wrap Cell Content:' button with a text icon. The result table has two columns: 'customer_id' and 'order_count'. It lists 19 customers with their respective order counts.

	customer_id	order_count
▶	1	12
	2	17
	3	17
	4	9
	5	14
	6	16
	7	13
	8	10
	9	10
	10	13
	11	9
	12	11
	13	12
	14	15
	15	15
	16	16
	17	17
	18	9
	19	15

20	17
21	13
22	15
23	9
24	15
25	15
26	6
27	7
28	17
29	20
30	16

6. Find total sales amount made by each customer

Query:

```
SELECT customer_id, SUM(total_amount) AS total_sales
FROM orders
GROUP BY customer_id;
```

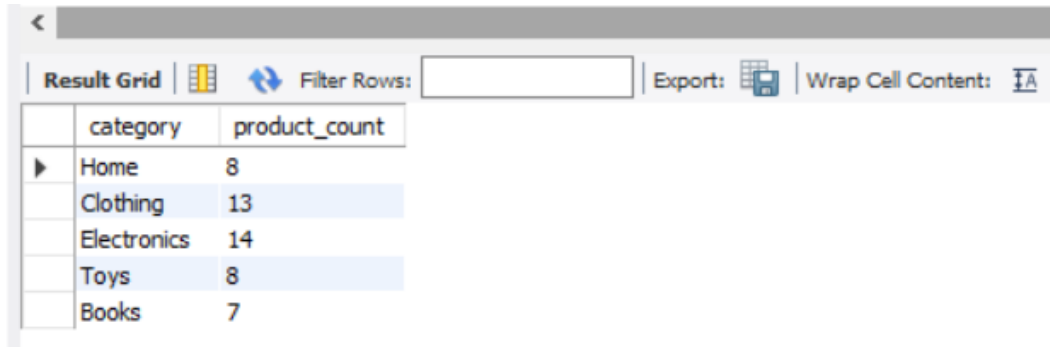
Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	customer_id	total_sales			
▶	1	183747.44			
	2	284420.07			
	3	253783.31			
	4	137562.22			
	5	262504.19			
	6	212173.58			
	7	167960.11			
	8	164701.78			
	9	106226.03			
	10	252722.75			
	11	173409.01			
	12	169653.79			
	13	152727.70			
	14	360324.18			
	15	260499.91			
	16	278469.73			
	17	262189.71			
	18	199823.08			
	19	222256.24			
	20	229435.29			
	21	205592.25			
	22	237879.99			
	23	167210.99			
	24	379286.82			
	25	335100.94			
	26	112039.80			
	27	109438.29			
	28	362584.19			
	29	435408.89			
	30	281841.38			

7. List the number of products sold per category

This helps category managers assess performance by department.

Query:

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category;
```



The screenshot shows a database query result grid with a toolbar at the top. The toolbar includes a 'Result Grid' button, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' button. The table below has two columns: 'category' and 'product_count'. The data rows are: Home (8), Clothing (13), Electronics (14), Toys (8), and Books (7).

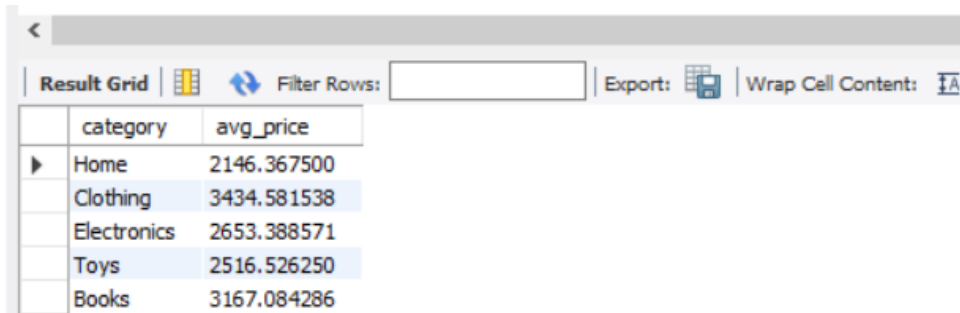
category	product_count
Home	8
Clothing	13
Electronics	14
Toys	8
Books	7

8. Find the average item price per category

Useful to compare pricing across departments.

Query:

```
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category;
```



The screenshot shows a database query result grid with a toolbar at the top. The toolbar includes a 'Result Grid' button, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' button. The table below has two columns: 'category' and 'avg_price'. The data rows are: Home (2146.367500), Clothing (3434.581538), Electronics (2653.388571), Toys (2516.526250), and Books (3167.084286).

category	avg_price
Home	2146.367500
Clothing	3434.581538
Electronics	2653.388571
Toys	2516.526250
Books	3167.084286

9. Show number of orders placed per day

Used to track daily business activity and demand trends.

Query:

```
SELECT DATE(order_date) AS order_date, COUNT(*) AS orders_per_day
FROM orders
GROUP BY DATE(order_date);
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
order_date	orders_per_day		
2025-03-02	3		
2024-10-09	2		
2025-05-08	2		
2024-09-19	3		
2025-04-08	3		
2024-10-25	3		
2024-07-29	2		
2024-07-30	2		
2025-06-10	2		
2025-02-16	2		
2025-03-11	3		
2025-01-15	4		
2025-05-12	2		
2024-09-24	1		
2024-07-18	1		
2024-09-09	3		
2024-08-19	1		
2024-06-15	1		
2024-07-06	2		
Result Grid	Filter Rows:	Export:	Wrap Cell Content:
order_date	orders_per_day		
2024-08-24	3		
2025-03-12	1		
2025-02-21	2		
2024-06-19	3		
2024-07-24	1		
2024-11-22	2		
2024-10-19	1		
2025-02-17	1		
2024-10-18	2		
2024-12-30	2		
2024-10-20	2		
2025-04-27	2		
2025-05-09	1		
2024-11-27	2		
2024-07-08	2		
2025-05-10	2		
2024-07-05	3		
2024-10-11	1		
2024-09-11	2		

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	order_date	orders_per_day
	2024-07-04	1
	2025-05-25	1
	2024-11-03	2
	2024-11-23	2
	2024-07-21	2
	2025-02-24	3
	2025-03-24	2
	2025-02-22	1
	2024-09-04	3
	2024-07-16	2
	2025-02-20	1
	2024-10-15	2
	2024-08-05	2
	2024-06-23	2
	2025-04-16	1
	2025-05-05	4
	2024-12-26	2
	2025-06-13	2
	2025-02-04	2

..... I am pasting a few outputs because the full output is too lengthy.

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	order_date	orders_per_day
	2025-03-31	
	2025-01-16	
	2024-10-23	1
	2024-12-15	1
	2024-07-14	1
	2025-04-29	1
	2024-11-13	1
	2025-01-31	1
	2024-10-10	1
	2024-11-15	1
	2025-03-14	1
	2024-12-25	1
	2025-02-28	1
	2024-07-20	1
	2024-12-28	1
	2024-11-12	1
	2025-06-05	1
	2024-11-07	1
	2025-02-08	1

10. List total payments received per payment method

Helps the finance team understand preferred transaction modes.

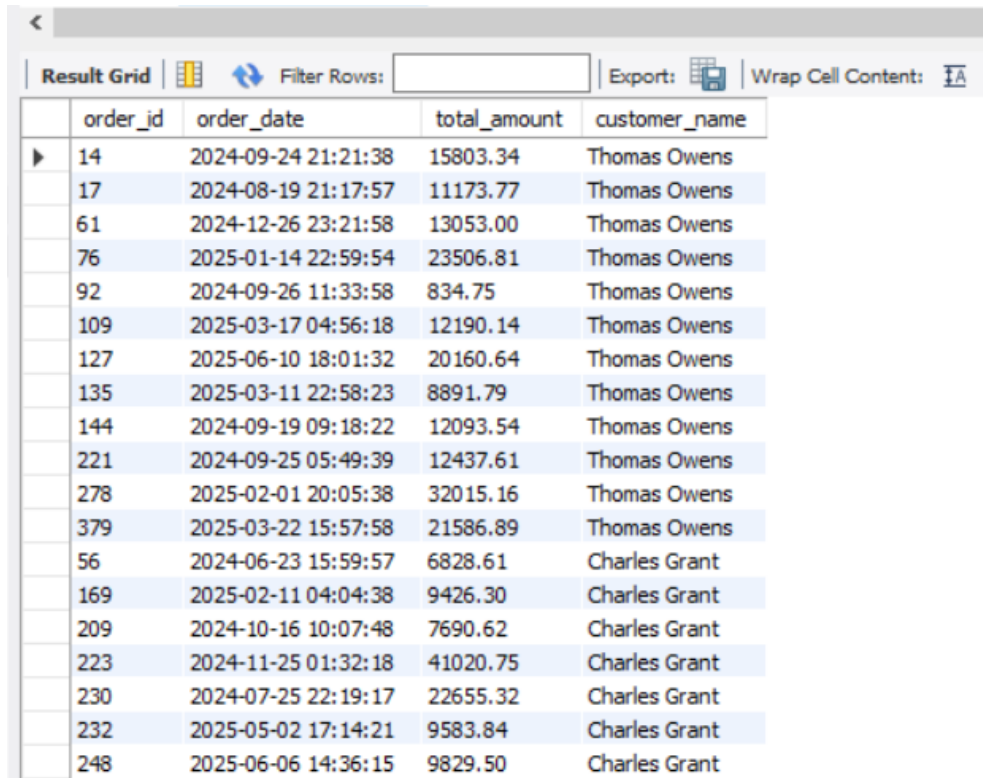
Answer: Cannot calculate because payment method data is not stored.

Level 4: Multi-Table Queries (JOINS)

1. Retrieve order details along with the customer name (INNER JOIN)

Used for displaying which customer placed each order.

```
SELECT o.order_id, o.order_date, o.total_amount, c.name AS customer_name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```



The screenshot shows a database query result grid. The grid has a toolbar at the top with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with 5 columns: 'order_id', 'order_date', 'total_amount', and 'customer_name'. The table contains 20 rows of data, showing orders placed by two customers: Thomas Owens and Charles Grant. The orders are sorted by order_id.

	order_id	order_date	total_amount	customer_name
▶	14	2024-09-24 21:21:38	15803.34	Thomas Owens
	17	2024-08-19 21:17:57	11173.77	Thomas Owens
	61	2024-12-26 23:21:58	13053.00	Thomas Owens
	76	2025-01-14 22:59:54	23506.81	Thomas Owens
	92	2024-09-26 11:33:58	834.75	Thomas Owens
	109	2025-03-17 04:56:18	12190.14	Thomas Owens
	127	2025-06-10 18:01:32	20160.64	Thomas Owens
	135	2025-03-11 22:58:23	8891.79	Thomas Owens
	144	2024-09-19 09:18:22	12093.54	Thomas Owens
	221	2024-09-25 05:49:39	12437.61	Thomas Owens
	278	2025-02-01 20:05:38	32015.16	Thomas Owens
	379	2025-03-22 15:57:58	21586.89	Thomas Owens
	56	2024-06-23 15:59:57	6828.61	Charles Grant
	169	2025-02-11 04:04:38	9426.30	Charles Grant
	209	2024-10-16 10:07:48	7690.62	Charles Grant
	223	2024-11-25 01:32:18	41020.75	Charles Grant
	230	2024-07-25 22:19:17	22655.32	Charles Grant
	232	2025-05-02 17:14:21	9583.84	Charles Grant
	248	2025-06-06 14:36:15	9829.50	Charles Grant

.... I am pasting a few outputs because the full output is too lengthy.

Result Grid		Filter Rows:		Export:		Wrap Cell Content:	
	order_id	order_date	total_amount	customer_name			
	345	2025-01-24 11:50:38	30411.52	Kara Zavala			
	347	2024-09-18 15:00:39	1429.45	Kara Zavala			
	362	2024-07-09 14:37:21	27540.77	Kara Zavala			
	38	2024-07-05 16:35:13	14681.26	Victoria Acevedo			
	39	2024-10-11 05:40:05	15439.15	Victoria Acevedo			
	52	2025-01-15 13:59:48	30529.35	Victoria Acevedo			
	104	2024-11-23 09:00:53	23699.22	Victoria Acevedo			
	147	2025-03-11 15:03:24	22594.47	Victoria Acevedo			
	175	2024-06-21 00:47:44	15372.41	Victoria Acevedo			
	197	2024-08-20 23:13:44	10668.23	Victoria Acevedo			
	199	2024-06-18 19:01:34	4993.36	Victoria Acevedo			
	203	2024-12-20 17:04:55	28369.14	Victoria Acevedo			
	245	2024-07-26 13:46:40	2496.68	Victoria Acevedo			
	259	2025-04-30 17:37:58	12043.33	Victoria Acevedo			
	275	2024-08-08 15:47:05	15495.63	Victoria Acevedo			
	277	2025-05-06 23:14:52	18188.67	Victoria Acevedo			
	289	2024-08-22 18:51:11	985.19	Victoria Acevedo			
	303	2025-05-05 23:51:29	31523.38	Victoria Acevedo			
	368	2024-11-13 12:51:48	34761.91	Victoria Acevedo			

2. Get list of products that have been sold (INNER JOIN with order_items)

Used to find which products were actually included in orders.

Answer: order_items table doesn't exist, so cannot produce output.

3. List all orders with their payment method (INNER JOIN)

Used by finance or audit teams to see how each order was paid for.

Answer: payments table / payment_method column doesn't exist.

4. Get list of customers and their orders (LEFT JOIN)

Used to find all customers and see who has or hasn't placed orders.

```
SELECT c.customer_id, c.name, o.order_id, o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
customer_id	name	order_id	total_amount
1	Thomas Owens	14	15803.34
1	Thomas Owens	17	11173.77
1	Thomas Owens	61	13053.00
1	Thomas Owens	76	23506.81
1	Thomas Owens	92	834.75
1	Thomas Owens	109	12190.14
1	Thomas Owens	127	20160.64
1	Thomas Owens	135	8891.79
1	Thomas Owens	144	12093.54
1	Thomas Owens	221	12437.61
1	Thomas Owens	278	32015.16
1	Thomas Owens	379	21586.89
2	Charles Grant	56	6828.61
2	Charles Grant	169	9426.30
2	Charles Grant	209	7690.62
2	Charles Grant	223	41020.75
2	Charles Grant	230	22655.32
2	Charles Grant	232	9583.84
2	Charles Grant	248	9829.50

... I am pasting a few outputs because the full output is too lengthy.

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
customer_id	name	order_id	total_amount
29	Kara Zavala	345	30411.52
29	Kara Zavala	347	1429.45
29	Kara Zavala	362	27540.77
30	Victoria Acevedo	38	14681.26
30	Victoria Acevedo	39	15439.15
30	Victoria Acevedo	52	30529.35
30	Victoria Acevedo	104	23699.22
30	Victoria Acevedo	147	22594.47
30	Victoria Acevedo	175	15372.41
30	Victoria Acevedo	197	10668.23
30	Victoria Acevedo	199	4993.36
30	Victoria Acevedo	203	28369.14
30	Victoria Acevedo	245	2496.68
30	Victoria Acevedo	259	12043.33
30	Victoria Acevedo	275	15495.63
30	Victoria Acevedo	277	18188.67
30	Victoria Acevedo	289	985.19
30	Victoria Acevedo	303	31523.38
30	Victoria Acevedo	368	34761.91

5. List all products along with order item quantity (LEFT JOIN)

Useful for inventory teams to track what sold and what hasn't.

Answer: order_items table doesn't exist, so quantity per product is not known.

6. List all payments including those with no matching orders (RIGHT JOIN)

Rare but used when ensuring all payments are mapped correctly.

Answer: Payments table does not exist.

7. Combine data from three tables: customer, order, and payment

Used for detailed transaction reports.

Answer: Only customers and orders exist, payments are missing.

Level 5: Subqueries (Inner Queries)

1. List all products priced above the average product price

Used by pricing analysts to identify premium-priced products.

```
SELECT *  
FROM products  
WHERE price > (SELECT AVG(price) FROM products);
```

Result Grid						
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:
	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
	22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
	24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
	25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
	26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
	29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
	30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28
	33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
	35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
	38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
	39	Data Add	Books	4063.38	62	2025-05-26 23:39:16
	41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
	42	Weight Enter	Clothing	3875.18	60	2024-01-26 08:21:55
	45	Suffer We	Electronics	3657.43	113	2024-07-25 12:40:57
	46	Age Treatment	Home	3728.83	160	2023-12-01 10:51:52
	48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
	49	Rest Something	Books	3077.82	181	2023-08-09 03:42:42

2. Find customers who have placed at least one order

Used to identify active customers for loyalty campaigns.

```
SELECT *  
FROM customers  
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	customer_id	name	email	phone	created_at				
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12				
	2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24				
	3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22				
	4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38				
	5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00				
	6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59				
	7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43				
	8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04				
	9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51				
	10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43				
	11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30				
	12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50				
	13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35				
	14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28				
	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42				
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50				
	17	Randy Mooney	user17@example.com	(158)927-7313x3	2023-09-21 13:23:02				
	18	Jeffrey Bray	user18@example.com	164.962.0222x92	2024-07-02 19:51:55				
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15				
	20	Megan Lee	user20@example.com	7044478333	2024-07-09 16:13:14				
	21	Jessica Zamora	user21@example.com	177-816-7773x00	2023-11-11 04:46:03				
	22	Peter Phillips	user22@example.com	389-656-1695	2025-05-22 06:12:06				
	23	Kathryn Mathews	user23@example.com	001-707-722-178	2023-12-21 15:30:24				
	24	Brandy Wright	user24@example.com	853.520.8915x61	2024-07-09 17:09:19				
	25	Cindy Hart	user25@example.com	617.574.8421x41	2025-05-27 18:02:54				
	26	Victoria Hall	user26@example.com	926-840-0016	2024-03-08 16:43:52				
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29				
	28	Joseph Stuart	user28@example.com	363-045-4287	2023-09-14 05:52:25				
	29	Kara Zavala	user29@example.com	434-985-5776x61	2023-09-27 15:57:09				
	30	Victoria Acevedo	user30@example.com	+1-325-063-7244	2023-11-28 12:30:59				

3. Show orders whose total amount is above the average for that customer

Used to detect unusually high purchases per customer.

```
SELECT *
FROM orders o
WHERE total_amount >
      (SELECT AVG(total_amount)
       FROM orders
       WHERE customer_id = o.customer_id);
```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
order_id	customer_id	order_date	status	total_amount
6	29	2024-10-25 07:33:59	Cancelled	31050.17
8	19	2024-07-30 22:49:49	Cancelled	32714.06
9	6	2025-06-10 17:00:25	Pending	24219.20
10	28	2025-02-16 12:45:59	Delivered	24342.52
14	1	2024-09-24 21:21:38	Shipped	15803.34
16	5	2024-09-09 03:00:40	Shipped	22856.73
18	13	2024-06-15 12:57:04	Cancelled	32001.24
21	22	2025-03-12 18:53:46	Pending	25364.11
22	20	2025-02-21 11:21:06	Shipped	21281.44
24	6	2024-06-19 07:09:04	Pending	22882.19
25	17	2024-07-24 12:56:24	Pending	23749.66
26	24	2025-02-16 23:16:49	Delivered	35167.92
29	7	2025-02-17 18:03:18	Delivered	26308.25
30	23	2024-10-18 13:37:49	Delivered	18628.74
32	15	2024-10-20 06:47:38	Pending	23536.35
33	8	2025-04-27 01:26:11	Pending	36147.09
36	25	2024-07-08 07:43:18	Cancelled	34125.89
40	26	2024-09-11 08:45:47	Shipped	24649.49
42	3	2025-05-25 05:16:38	Cancelled	21622.22

.... I am pasting a few outputs because the full output is too lengthy.

order_id	customer_id	order_date	status	total_amount
368	30	2024-11-13 12:51:48	Delivered	34761.91
369	3	2024-09-04 02:09:09	Shipped	33503.41
370	16	2025-01-31 05:25:11	Delivered	22871.46
372	7	2024-10-10 21:32:16	Shipped	18814.53
373	19	2024-09-11 14:32:08	Delivered	29373.72
379	1	2025-03-22 15:57:58	Cancelled	21586.89
383	6	2024-07-05 13:30:29	Delivered	18148.12
386	4	2024-12-18 11:35:41	Cancelled	25747.34
388	19	2025-03-08 00:36:03	Pending	17639.96
389	11	2025-01-01 08:16:36	Delivered	20299.18
390	7	2025-06-06 16:22:38	Delivered	18092.48
392	24	2024-11-12 21:00:20	Shipped	27858.02
393	28	2024-12-17 21:08:26	Pending	44504.56
394	2	2025-06-05 18:39:21	Delivered	18154.30
395	9	2025-02-11 06:20:16	Cancelled	12331.34
396	28	2024-07-21 04:35:22	Pending	35029.57
397	10	2024-11-10 15:30:55	Delivered	26820.10
399	17	2025-02-08 16:59:16	Shipped	19557.58

4. Display customers who haven't placed any orders

Used for re-engagement campaigns targeting inactive users.

```
SELECT *
FROM customers
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
```

customer_id	name	email	phone	created_at
NULL	NULL	NULL	NULL	NULL

5. Show products that were never ordered

Helps with inventory clearance decisions or product deactivation.

Answer: Product-level order data is not stored.

6. Show highest value order per customer

Used to identify the largest transaction made by each customer.

```
SELECT * FROM orders o
WHERE total_amount =
    (SELECT MAX(total_amount)
     FROM orders
     WHERE customer_id = o.customer_id);
```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
order_id	customer_id	order_date	status	total_amount
18	13	2024-06-15 12:57:04	Cancelled	32001.24
33	8	2025-04-27 01:26:11	Pending	36147.09
62	10	2025-06-13 23:21:01	Cancelled	35723.17
66	27	2024-06-19 09:41:16	Pending	26176.05
68	24	2025-03-15 07:27:20	Delivered	36159.92
80	12	2025-03-28 18:42:12	Delivered	38656.26
107	14	2025-02-01 02:02:18	Shipped	48042.06
121	16	2024-11-21 16:50:01	Pending	37415.67
128	7	2025-05-05 16:49:18	Pending	28589.04
133	11	2024-11-20 00:44:46	Shipped	37129.82
139	18	2025-04-13 20:12:26	Delivered	39857.19
174	19	2024-09-30 11:32:38	Cancelled	35676.00
189	25	2024-12-17 18:53:05	Shipped	40872.29
196	21	2025-02-07 10:13:15	Shipped	24096.64
198	22	2024-11-11 00:17:56	Delivered	36504.68
243	15	2024-12-19 09:20:09	Delivered	40510.54
256	17	2025-06-08 16:15:49	Delivered	40944.10
258	26	2025-04-18 05:53:20	Pending	38836.44
266	5	2025-03-21 21:25:00	Delivered	39921.78
267	23	2024-07-03 02:51:59	Shipped	33419.96
278	1	2025-02-01 20:05:38	Delivered	32015.16
281	3	2024-11-08 07:40:55	Shipped	41679.11
309	29	2024-09-13 13:25:58	Delivered	43867.08
315	2	2025-01-27 15:23:27	Cancelled	42056.04
327	20	2025-05-05 23:49:05	Shipped	39563.51
330	6	2025-01-10 15:50:45	Pending	39003.19
348	9	2024-12-29 18:35:10	Cancelled	21414.44
368	30	2024-11-13 12:51:48	Delivered	34761.91
386	4	2024-12-18 11:35:41	Cancelled	25747.34
393	28	2024-12-17 21:08:26	Pending	44504.56

7. Highest Order Per Customer (Including Names)

Used to identify the largest transaction made by each customer. Outputs name as well.

```
SELECT c.name, o.order_id, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total_amount =
      (SELECT MAX(total_amount)
       FROM orders
       WHERE customer_id = c.customer_id);
```

	name	order_id	total_amount
▶	Thomas Owens	278	32015.16
	Charles Grant	315	42056.04
	Kaitlin Richards	281	41679.11
	Christina Williams	386	25747.34
	David Allen	266	39921.78
	Mark Duke	330	39003.19
	Briana Wright	128	28589.04
	John Bryan	33	36147.09
	Jason Thompson	348	21414.44
	Shawn Hill	62	35723.17
	Walter Jenkins	133	37129.82
	Mary Knight	80	38656.26
	Leslie Wilson	18	32001.24
	Deborah Arias	107	48042.06
	Austin Flores	243	40510.54
	Amy Landry	121	37415.67
	Randy Mooney	256	40944.10
	Jeffrey Bray	139	39857.19
	Amanda Bright	174	35676.00
	Megan Lee	327	39563.51
	Jessica Zamora	196	24096.64
	Peter Phillips	198	36504.68
	Kathryn Mathews	267	33419.96
	Brandy Wright	68	36159.92
	Cindy Hart	189	40872.29
	Victoria Hall	258	38836.44
	Adrienne Green	66	26176.05
	Joseph Stuart	393	44504.56
	Kara Zavala	309	43867.08
	Victoria Acevedo	368	34761.91

Level 6: Set Operations

1. **List all customers who have either placed an order or written a product review**

Used to identify engaged customers from different activity areas.

Answer: cannot be done bcz — reviews table is missing.

2. **List all customers who have placed an order as well as reviewed a product [intersect not supported]**

Used to identify highly engaged customers for rewards.

Answer: cannot be done bcz — reviews table is missing.